技术图书排版

杜金房　著

INVITE sip:you@xswitch.cn SIP/2.0
Via: SIP/2.0/TLS xswitch.cn:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: "Seven Du" <sip:seven@xswitch.cn>;tag=9fxced76sl
To: You <sip:you@xswitch.cn>
Call-ID: 3848276298220188511@xswitch.cn
CSeq: 2 INVITE
Contact: <sip:seven@xswitch.cn;transport=tls>

小樱桃出品

技术图书排版

杜金房

版权所有　　侵权必究

图书不在版编目（NCIP）数据

技术图书排版/杜金房　著/2019.7
ISBN 7-DU-777777-7

本书分享写作 FreeSWITCH 相关图书过程中总结的一些排版经验，也使用了类似《FreeSWITCH 文集》的封面。本书所有源代码都在 Github 上，书中有相关链接。

技术图书排版

作　　者	杜金房
封面设计	杜金房
校　　对	高　斐
排　　版	桃　子　　拾　得
责任编辑	吕佳娉
开　　本	90 mm × 160 mm
印　　张	7.5
印　　数	7
版　　数	2019 年 7 月第 1 版　2019 年 7 月第 1 次发布
电子邮箱	freeswitch@dujinfang.com

版权所有　　侵权必究

前　言

我出过一本书——《FreeSWITCH 权威指南》，也写过很多 FreeSWITCH 相关的电子书。有读者问我是怎么排版的，不揣鄙陋，愿与大家分享。

我最初写博客都是使用的是 Markdown[1]，后来写书也使用 Markdown 做简单排版。但一到出版社编辑那儿，就必须用 Word 了。也跟出版社聊过，是否可以用 Latex 排版，但出版社的答复是养一个 Latex 排版师太贵了，因此，作者们还是需要使用 Word 排版。

对于那些写小说或故事的作者，或许用 Word 排版还是不错的，但是，对于像我这样的技术人员，由于书中有很多代码和图表，用 Word 排版就比较累，尤其是技术人员好多都在用 Mac，与 Windows 版的 Word 兼容性还比较差，写起来就更痛苦了。

[1] https://en.wikipedia.org/wiki/Markdown

使用 Markdown 格式写作，比 Word 要轻松多了，同时，使用一些辅助工具也能做到比较好的排版。如果万一有一天写出来的书能够正式出版，也希望 Word 是最后一步，把痛苦留到最后。

Word 有一个功能确实不大好替代，那就是『修订』功能，使用它可以让作者和编辑很方便地交互修改文件。但文件传来传去也很烦。不知道一些在线的协作工具如石墨文档等，是否适合这种协作。

当然，其实作者不应该关注排版，而是在保证内容正确的前提下，把章节、强调、引用、代码之类的都标注出来即可，出版社是有专人进行排版的。而做这些标注，Markdown 就够了。而且 Markdown 文件可以很方便地放到 Git 仓库中。我们不期望出版社所有的编辑也能熟悉 Git，但如果真有哪天编辑们也喜欢 Git 了，那社会就真的进步了。

关于排版，本书主要讲一下相关的模板，以及一些排版原则：

- 支持标准的 Markdown，暂不支持各种扩展；
- 使用开源字体；
- 代码应该有单独的格式，用等宽字体，最好支持语法高亮；
- 应该有移动版，目前大多数 PDF 并不适合在手机

上阅读；

- 移动版不应该首行缩进，因为页面太窄，缩进反而影响阅读体验；
- 标准版应该首行缩进；
- 印刷版应该奇偶页不同；
- 提供 Word 版方便与其它人交流；
- 还可以生成 EPUB 电子书等其它格式，欢迎提 pr。

本书是可以自举的，也就是说你可以通过本书的源代码生成本书的 PDF。参见：

https://github.com/seven1240/latex

最后，本书写作的目的并不是教你成为一个排版专家，相反，希望通过分享我的排版模板，让广大技术人员专注于用自己喜欢的工具写好自己的作品，忘记排版;)。

杜金房/2019/烟台

目　录

前　言 III

1 技术图书排版 1

1.1 Makefile 2

1.2 meta.md 8

1.3 cover.tex 8

1.4 cover-std.tex 12

1.5 header.tex 14

2 主模板 16

3 我在 Mac 上的写作工具链 37

写在最后 **50**

作者简介 **52**

版权声明 **53**

广告 **54**

关于广告的广告 54

XSWITCH 54

技术支持 55

FreeSWITCH-CN 公众号 56

FreeSWITCH 相关图书 56

知识星球 57

59

第一章 技术图书排版

欢迎来到排版世界。我们用 Markdown 格式写作，用 LaTex 模板排版，用 Pandoc 做格式转换。

Markdown 是一个文档格式，它是基于纯文本的，通过简单的格式约定，既做到源文件易读，又做到可以支持一些基本的格式。本文就是用 Markdown 写成的。

Pandoc[1]是一个文档转换工具，是一个瑞士军刀。它可以在各种文档格式间转换，在此我们会将我们的 Markdown 文件转换成 PDF。

LaTex[2]是世界上最先进的排版系统，在文档转换过程中，我们会用到 LaTex。推荐安装 TexLive[3]，但完整版安装包有 4 个多 G，太大了，所以一般安装 BasicTex 就够

[1] https://pandoc.org/

[2] https://www.latex-project.org/

[3] http://www.tug.org/mactex/morepackages.html

了，这个版本比较小，但宏包不全，遇到有缺少的宏包可以后续用 `tlmgr install 包名` 命令安装。

本书是在 Mac 上编译的，笔者也制作了个 Docker 镜象方便大家使用，参见下一节。

这是本书的第一章。到此，章节、段落、脚注、链接等写法你都已经看到了（当然，如果你阅读的是 PDF 的话，你需要看一下源文件）。下面，我们看下本书中用到的一些文件。

1.1 Makefile

关于 Pandoc 的用法我们不会详细解释，感兴趣的应该去看官方网站上的文档，我们只是解释一下我用到的一些命令。

先看 `Makefile`。好吧，我们需要先学习一下 Makefile。

Makefile 不是必需的，但作为一名程序员，它是一个很方便使用的工具。

首先定义了一个变量，`PANDOC` 就指向 `pandoc` 可执行文件，如果在不同的系统上使用，可以更新路径。

`all` 定义了一个目标（Target），如果在命令行上执行 `make` 命令，就默认使用这个目标。可以看到，它其

实**依赖**于另外几个目标 `mobile`、 `book` 和 `print`，后面我们会讲到这些目标，在命令行上也可以单独 `make` 一个目标，如 `make mobile`。

`VER` 只是一个版本号。 `SRC` 为本书全部的源文件。

```
PANDOC := pandoc

all: mobile book print

VER=7

SRC = meta.md \
    README.md \
    postface.md
```

先来个小目标;)， `preface.tex` 是一个小目标，是『前言』部分。因为我们希望前言能在目录的前面，所以我们需要一个 `tex` 文件，但我们还是想用 Markdown 格式写，所以，我们会把 `preface.md` 转换成 `tex` 并插入到文档相应的位置。

其中 `-s` 为 Smart 的意思（嗯，欲知详情看官方文档）， `--variable` 为变量，设置 LaTex 的文档格式，后面我们会有模板文件中看到。 `--template` 选择一个模板，在此，我们自创了一个空模板，它会生成不带模板

的 LaTex 文件。好吧，如果听不明白也没关系，你可以看一下生成的这个文件的内容，初步了解一下 LaTex 的格式。

```
preface.tex: preface.md
    $(PANDOC) -s --variable documentclass=report \
    --template template-dummy.tex \
    -o preface.tex preface.md
```

`out` 是一个目标，这就是 Makefile 的魔术，如果没有这个目标文件夹，就执行下面的 `mkdir` 命令创建它。再次说明，Pandoc 并不依赖于 Makefile，你可以手工执行命令创建这个文件夹，但是我们使用 Makefile 只是为了方便。

```
out:
    mkdir out
```

`book` 是我们标准的目标，它会生成一个 PDF 文件（`-o` 指定）。`--toc` 是 `Table of Content`，即自动生成图书目录。这里的 `template.tex` 是我们的模板文件，后面我们还会详细讲。`--number-sections` 自动生成章节

号。 pdf-engine 我们选 xelatex ，对中文比较友好，另一个选项是 pdflatex ，但对中文支持稍差点。 --include-xxx 表示将相关文件插入到模板文件相应的位置。

```
book: out preface.tex $(SRC)
    $(PANDOC) -s --toc \
    --template template.tex \
    --number-sections \
    --pdf-engine=xelatex \
    --include-in-header=cover-std.tex \
    --include-before=header.tex \
    --include-before-body=preface.tex \
    -o out/book-标准版-$(VER).pdf \
    $(SRC)
```

标准版是 A4 的纸张，在手机上显示不适合阅读，我们生成个移动版的，设置纸张大小为 9 x 16cm ，即对于 16:9 的手机屏幕，刚好显示一页。在此，我们传入了一个 mobile=true 参数。

```
mobile: out preface.tex $(SRC)
    $(PANDOC) -s --toc \
    --template template.tex \
    --number-sections \
    --pdf-engine=xelatex \
```

```
    --variable mobile=true \
    --include-in-header=cover.tex \
    --include-before=header.tex \
    --include-before-body=preface.tex \
    -o out/book-移动版-$(VER).pdf \
    $(SRC)
```

图书是要出版的，一般出版的尺寸不会是 A4 的，所以要设置不同的尺寸，另外，出版图书跟电脑版还有一个重要的区别是奇偶页不同（页边距和页码位置等），我们加了个 `print` 变量控制打印的格式。看了这么多年书，你有没有注意到这个问题呢？

```
print: out preface.tex $(SRC)
    $(PANDOC) -s --toc \
    --variable print=true \
    --variable fontsize=11pt \
    --template template.tex \
    --number-sections \
    --pdf-engine=xelatex \
    --include-in-header=cover-dummy.tex \
    --include-before=header.tex \
    --include-before-body=preface.tex \
    -o out/book-印刷版-$(VER).pdf \
    $(SRC)
```

为了照顾那些顽固地想看 Word 版的人，我们增加了一个 `make docx`，就可以直接生成 Word 版了。其中 `reference.docx` 是一个模板，我在默认模板的基础上稍稍改了下增加了章节号。你可以参考 Pandoc 的官方文档看还能否玩出什么花来。

```
docx: out preface.tex $(SRC)
    $(PANDOC) -s --toc \
    --number-sections \
    --reference-doc reference.docx \
    -o out/技术图书排版-$(VER).docx \
    preface.md $(SRC)
```

如果你本地没有安装 Pandoc 以及 LaTex 环境，可以使用笔者制作的 Docker 镜象。在命令行上执行 `make docker` 会进入一个 Docker 环境，并把当前目录映射到 `/team` 目录中，然后就可以继续 `make` 生成 PDF 了。

```
docker:
  docker run -it -v `pwd`:/team dujinfang/
↪  texlive_pandoc:latest bash
```

1.2 meta.md

meta.md 里面定义了一些变量，YAML 格式。这些变量在主模板文件中会用到。

```
---
documentclass: report
title: 技术图书排版
author: 杜金房
title-meta: 技术图书排版
author-meta: 杜金房
publisher: 版权所有\qquad 侵权必究
verbatim-in-note: true
---
```

1.3 cover.tex

我们先从这个文件熟悉一个 LaTex 的语法。你不需要精通 LaTex，但学一点总是有好处。

这是本书的封面，如果读到这里，你应该已经看到这个封面了。简单起见，我直接用了《FreeSWITCH 文集》的封面。虽然我们可以直接用个 PNG 或 JPEG 图片做

封面，但是作为一名程序员，我还是喜欢用代码生成封面，尽量少地依赖 PhotoShop 之类的软件。

我们使用 `tikz` 宏包，嗯，它是在 LaTex 里画图用的。类似于程序语言中的模块，LaTex 使用宏包扩展本身的功能。其中 `%` 是注释。如果把 `texcoord` 那行注释去掉，可以看到一些参考线。

```
\usepackage{tikz}
\usepackage[absolute,overlay]{textpos}
% uncomment to see grid system
% \usepackage[texcoord,grid,gridcolor=red!
↪  10,subgridcolor=green!10,gridunit=cm]{eso-pic}
```

`shadowtext` 给『FreeSWITCH』画上阴影。

```
\usepackage{shadowtext}
\shadowcolor{black}
\shadowoffset{1pt}
```

下面定义了一个新命令 `\cover`，用于在正文中『画』封面。从 `yellow` 到 `Orange` 做个渐变的颜色背景。然后把颜色再切换成白色。

```latex
\newcommand{\cover}{
\begin{tikzpicture}[remember picture,overlay]
\path [top color = yellow, bottom color = Orange] (current page.south west) rectangle (current page.north east);
\end{tikzpicture}
\color{white}
```

设置这是一个『空』（empty）页面（告诉 LaTex 不需要自动生成页码之类的），居中，写上『FreeSWITCH』，加上 wenji.png，这是一个图版，然后写上作者。

```latex
\thispagestyle{empty}
\pagenumbering{gobble}
% \chapter*{}
\bigskip
\begin{center}
  \textbf{\fontsize{36}{48}\selectfont\shadowtext{FreeSWITCH}}
  \\[2em]
  \Oldincludegraphics[width=0.5\paperwidth]{wenji.png}
  \\[2em]
  \textbf{\large \theauthors \quad 著\\[2em]}
\end{center}
```

找一个位置，放上『小樱桃出品』，使用绝对坐标。

```
\begin{textblock*}{4cm}[0.5,0.5](0.5\paperwidth, 13.8cm)
\small
\color{white}
\center
\textbf{小樱桃出品}
\color{black}
\end{textblock*}
```

bigskip 会自动填充中间的空间，然后在下面画一个文本框放上一些装饰文本（笔者是做 SIP 通信的，因此放了一段 SIP 消息）。

```
\bigskip
\vfill
\begin{adjustwidth}{-2.3mm}{}
\mbox{\vbox{
\color{yellow}
\small
INVITE sip:you@xswitch.cn SIP/2.0\newline
Via: SIP/2.0/TLS xswitch.cn:5060;branch=z9hG4bK74bf9\newline
Max-Forwards: 70\newline
From: {\color{yellow!60!white}\Oldtexttt{"}Seven
↪  Du\Oldtexttt{"}}
↪  <sip:seven@xswitch.cn>;tag=9fxced76sl\newline
To: You <sip:you@xswitch.cn>\newline
Call-ID: 3848276298220188511@xswitch.cn\newline
```

```
CSeq: 2 INVITE\newline
Contact: <sip:seven@xswitch.cn;transport=tls>
}}
\end{adjustwidth}

\color{black}
}
```

LaTex 的语法比较奇怪，还是那句话，懂不懂没关系（因为模板我已经写好了啊，除非你要做自己的封面;)）。

不过瘾？下面再来一个。

1.4 cover-std.tex

这个文件其实跟 `cover.tex` 差不多，只是调整了一些尺寸，看看 `diff` 吧：

```
$ diff cover.tex cover-std.tex
8c8
< \shadowoffset{1pt}
---
> \shadowoffset{2pt}
21c21
```

```
<   \textbf{\fontsize{36}{48}\selectfont\shadowtext{技术图
↪   书排版}}
---
>   \textbf{\fontsize{72}{96}\selectfont\shadowtext{技术图
↪   书排版}}
23c23
<   \Oldincludegraphics[width=0.5\paperwidth]{img/
↪   wenji.png}
---
>   \Oldincludegraphics[width=0.4\paperwidth]{img/
↪   wenji.png}
25c25
<   \textbf{\large \theauthors \quad 著\\[2em]}
---
>   \textbf{\huge \theauthors \quad 著\\[2em]}
28,29c28,29
< \begin{textblock*}{4cm}[0.5,0.5](0.5\paperwidth, 13.8cm)
< \small
---
> \begin{textblock*}{4cm}[0.5,0.5](0.5\paperwidth, 24cm)
> \Large
38d37
< \begin{adjustwidth}{-2.3mm}{}
41c40
< \small
---
> \Large
51d49
< \end{adjustwidth}
```

1.5 header.tex

嗯，标准的书上都有个『图书在版编目（NCIP）数据』，我们的书还没有出版，就加上个『不』吧。

其中 tabular 是表格，其它的不多解释了吧，对照效果图自己看，哈哈。

```
\newpage
\pagecolor{white}
\thispagestyle{empty}
% \vspace*{0.5cm}
\noindent\textbf{图书不在版编目（NCIP）数据}

\vspace{1em}

\noindent\thetitle/\theauthor\quad 著/2019.7
\\
\noindent ISBN 7-DU-777777-7

% \vspace*{2cm}

\noindent {本书分享写作 FreeSWITCH 相关图书过程中总结的一些排
↪ 版经验，也使用了类似《FreeSWITCH 文集》的封面。}

\bigskip
\vfill
\noindent {\bf \thetitle}
```

```latex
\begin{adjustwidth}{-2.3mm}{}
\begin{tabular}{cl}
  \hline
  {\bf 作\qquad 者} & \theauthor\\
  {\bf 封面设计     } & \theauthor\\
  {\bf 校\qquad 对} & 高\quad 斐\\
  {\bf 排\qquad 版} & 桃\quad 子\qquad 拾\quad 得\\
  {\bf 责任编辑} & 吕佳娉\\
  {\bf 开\qquad 本} & \printlen[0][mm]{\paperwidth} ×
↪  \printlen[0][mm]{\paperheight}\\
  {\bf 印\qquad 张} & 7.5\\
  {\bf 印\qquad 数} & 7\\
  {\bf 版\qquad 数} & 2019 年 7 月第 1 版\quad 2019 年 7 月第 1
↪  次发布\\
  {\bf 电子邮箱}   &  freeswitch@dujinfang.com \\
  \hline
\end{tabular}
\end{adjustwidth}
\vfill
\begin{center}
  {\bf \thepublisher}
\end{center}
```

第二章 主模板

`template.tex` 是我们的主模板。之所以我们将封面等模板放到其它的文件里，是因为这样会便于用一套模板做不同的书。

Latex 文档的开头叫导言区（Preamble），定义了纸张及其它排版的参数。 `if-else` 之类的条件判断是 Pandoc 加上去的，它会根据命令行上输入的参数不同选择不同的设置，比如一般的图书用 `report`，而一篇文章则用 `article`。你也可以看到，如果有 `print` 参数生成可以印刷图书的话，那就会加上 `twoside,openright` 以支持双面打印，以及奇偶而不同之类。

```
\documentclass[$if(fontsize)$$fontsize$,$endif$$if(lang)$
↪  $lang$,$endif$CJKutf8$if(print)
↪  $,twoside,openright$endif$]{$documentclass$}
```

中文支持设定。

```
\XeTeXlinebreaklocale "zh"
\XeTeXlinebreakskip = 0pt plus 1pt minus 0.1pt
\hyphenation{FreeSWITCH}
\usepackage{xprintlen} % print paper length in mm
```

根据变量设置纸张大小。

```
$if(mobile)$
% \usepackage[showframe=true,papersize={9cm, 16cm},
↪  text={8.4cm, 14cm}]{geometry}
\usepackage[papersize={9cm, 16cm}, text={8.4cm, 14.1cm}]
↪  {geometry}
$else$
$if(print)$
\usepackage[papersize={19cm, 23.6cm},text={15cm, 19.6cm}]
↪  {geometry}
$else$
\usepackage[top=1in,bottom=1in,left=1.25in,right=1.25in]
↪  {geometry}
$endif$
$endif$
```

跟字体相关的设置。我们使用了谷歌的思源 CJK（中日

韩文）字体[1]。嗯，因为该字体是开源的。如果你使用其它字体嵌入到 PDF 中，要注意字体的版权问题。

mainfont 是主字体， monofont 是等宽字体， romanfont 就是英文相关的代码相关的字体，我们实验性地使用了 Adobe 的 Source Code Pro，因为它有斜体（Italic），注意中文是没有斜体的，我们用楷体代替。所有用到的字体都已经打包到了 Docker 镜象里，如果你不使用 Docker，你需要下载并安装这些字体。可以从它们的官方网站上找找，或者到https://github.com/seven1240/font上面找。

```
\usepackage{changepage}
\usepackage{float}
\usepackage{fontspec}
\newcommand\mainfont{Noto Sans CJK SC DemiLight}
\newcommand\kaifont{Adobe Kaiti Std}
\setmainfont[BoldFont=Noto Sans CJK SC
↪  Bold,ItalicFont={\kaifont}]{\mainfont}
\newfontfamily\kai{\kaifont}
\newfontfamily\zhfont[BoldFont=Noto Sans CJK SC
↪  Bold,ItalicFont={\kaifont}]{\mainfont}
\newfontfamily\zhpunctfont[BoldFont=Noto Sans CJK SC Bold]
↪  {\mainfont}
\setromanfont[Mapping=tex-text,BoldFont=Noto Sans CJK SC
↪  Bold,ItalicFont=Source Code Pro Italic]{\mainfont}
```

[1] https://www.google.com/get/noto/

```
\setmonofont{Noto Sans Mono CJK SC}
```

设置中文间距等。

```
\usepackage{zhspacing}
\zhspacing
```

longtable 支持跨页的表格，普通表格不能跨页。

```
\usepackage{longtable}
```

首行缩进。如果是手机端，就不缩进，因为手机屏幕比较小，缩进会不好看。

```
\usepackage{indentfirst}

$if(mobile)$
\setlength{\parindent}{0em}
$else$
\setlength{\parindent}{2em}
$endif$
```

fancyvrb是设置每页的页眉和页脚格式。注意\leftmark和rightmark是设成奇偶页不同的。

```
\usepackage{fancyvrb}
\DefineVerbatimEnvironment{verbatim}{Verbatim}
{frame=lines,
framerule=0.4pt,
baselinestretch=1,
fontfamily="Source Code Pro",
fontsize=\tiny,
xleftmargin=0pt,
xrightmargin=0pt,
rulecolor=\color{grey},
framesep=3mm,
numbers=left,
samepage=true
}
\fvset{frame=lines,framerule=0.4pt,rulecolor=\color{cyan},fram

\usepackage{fancyhdr}
\pagestyle{fancy}
\fancyhead{}
\fancyfoot{}
\fancyhead[LO,LE]{$title$}
\fancyfoot[C]{\small ------ \thepublisher ------}
\fancyfoot[RO, LE]{\thepage}
$if(article)$
$else$
% \fancyhead[LE,RO]{\chaptermark}
```

```
$endif$
% \fancyhead[LO,RE]{\sectionmark}
$if(mobile)$
$else$
\fancyhead[RO]{\nouppercase{\leftmark}}
\fancyhead[RE]{\nouppercase{\rightmark}}
\headheight 25pt
\headsep 10pt
$endif$
```

默认章节号是英文的，翻译成中文。

```
\renewcommand{\contentsname}{目\quad 录}
\renewcommand\listfigurename{插图目录}
\renewcommand\listtablename{表格目录}
% \renewcommand\refname{参考文献}
\renewcommand\indexname{索引}
\renewcommand\figurename{图}
\renewcommand\tablename{表}
\renewcommand\abstractname{摘要}
\renewcommand\partname{第\,\thepart\,部分}
\renewcommand\appendixname{附录}
\renewcommand\today{\number\year 年\number\month
↪  月\number\day 日}
```

```
\providecommand{\CJKnumber}[1]
↪ {\ifcase#1\or{一}\or{二}\or{三}\or{四}\or{五}\or{六}\or{七
↪ 一}\or{十二}\or{十三}\or{十四}\or{十五}\or{十六}\or{十
↪ 七}\or{十八}\or{十九}\or{二十}\or{二十一}\or{二十
↪ 二}\or{二十三}\or{二十四}\or{二十五}\or{二十六}\or{二十
↪ 七}\or{二十八}\or{二十九}\or{三十}\fi}

\usepackage{titlesec}
\titleformat{\chapter}{\centering\LARGE\bfseries}
↪ {\textbf{第\CJKnumber{\thechapter}章}}{0.5em}{}
$if(article)$
$else$
\renewcommand{\chaptername}{第\CJKnumber{\thechapter}章}
$endif$

\titleformat{\part}{\centering\Huge}{第\,\thepart\,部分}
↪ {1em}{}
\renewcommand\partname{第\,\thepart\,部分}
```

这一段设置不知道是否起作用，还是页眉页脚相关的。

```
\usepackage{fancyvrb}
\fvset{fontsize=\footnotesize}
% \fvset{xleftmargin=0.8cm}
\fvset{frame=lines,framerule=0.4pt,rulecolor=\color{cyan},fram
% \fvset{commandchars=\\\{\}}
```

Verbatim 是代码段的环境。

```
\RecustomVerbatimEnvironment{verbatim}{Verbatim}{}
```

设置一些颜色。

```
\usepackage[usenames, dvipsnames]{xcolor}
\definecolor{mygray}{gray}{0.9}
\definecolor{darkblue}{rgb}{0.0, 0.0, 0.61}
\definecolor{indigo}{rgb}{0.29, 0.0, 0.51}
\definecolor{navyblue}{rgb}{0.0, 0.0, 0.5}
\definecolor{NAVYBLUE}{rgb}{0.0, 0.0, 0.5}
\definecolor{myyellow}{RGB}{255,255,0}
\definecolor{thegray}{RGB}{60,60,60}
\definecolor{darkblue}{RGB}{0,0,139}
\definecolor{lime}{RGB}{0,255,0}
\definecolor{wireshark}{RGB}{0,153,204}
\definecolor{wireshark1}{RGB}{102,204,255}
\definecolor{BLACK}{rgb}{0.0, 0.0, 0.0}
\definecolor{darkgreen}{RGB}{0,100,0}
```

> 设置引用相关的字体和颜色，比如本段就是一段引用，源文件中以 > 开头，注意到了吗？

```
\newfontfamily\quotefont{STKaiti}
\let\quoteOLD\quote
\def\quote{\quoteOLD\color{thegray}
↪  \small\quotefont\selectfont}
```

奇偶页不同的设置。

```
$if(print)$
\let\tmp\oddsidemargin
\let\oddsidemargin\evensidemargin
\let\evensidemargin\tmp
\reversemarginpar
$endif$
```

还是代码段环境的一些设置。

```
% for inline code
\let\Oldtexttt\texttt
\renewcommand{\texttt}[1]{\Oldtexttt{\,\color{navyblue}
↪  #1\color{black}\,}}
```

数学公式相关。

```
% \usepackage{lmodern}
\usepackage{amssymb,amsmath}
```

这是 Pandoc 提供的模板的一些默认设置，没改，也没有深入研究。

```
\usepackage{ifxetex,ifluatex}
\usepackage{fixltx2e} % provides \textsubscript
% use microtype if available
\IfFileExists{microtype.sty}{\usepackage{microtype}}{}
\ifnum 0\ifxetex 1\fi\ifluatex 1\fi=0 % if pdftex
  \usepackage[utf8]{inputenc}
$if(euro)$
  \usepackage{eurosym}
$endif$
\else % if luatex or xelatex
  \usepackage{fontspec}
  \ifxetex
    \usepackage{xltxtra,xunicode}
  \fi

  ↪  \defaultfontfeatures{Mapping=tex-text,Scale=MatchLowerca
  \newcommand{\euro}{€}
$if(mainfont)$
    \setmainfont{$mainfont$}
$endif$
```

```
$if(sansfont)$
    \setsansfont{$sansfont$}
$endif$
$if(monofont)$
    \setmonofont{$monofont$}
$endif$
$if(mathfont)$
    \setmathfont{$mathfont$}
$endif$
\fi
$if(geometry)$
\usepackage[$for(geometry)$$geometry$$sep$,$endfor$]
↪ {geometry}
$endif$
$if(natbib)$
\usepackage{natbib}
\bibliographystyle{plainnat}
$endif$
$if(biblatex)$
\usepackage{biblatex}
$if(biblio-files)$
\bibliography{$biblio-files$}
$endif$
$endif$
```

---

listings 也是代码段环境，但我们好像没有用到（或者是以前用到会有什么问题），如果在命令行参数中使用 --listings 就可以启用它。

```
$if(listings)$
\usepackage{listings}
\lstset{ % General setup for the package
  language=perl,
  basicstyle=\small\sffamily,
  numbers=left,
  numberstyle=\tiny,
  frame=tb,
  tabsize=4,
  columns=fixed,
  showstringspaces=false,
  showtabs=false,
  keepspaces,
  commentstyle=\color{red},
  keywordstyle=\color{blue},
  backgroundcolor=\color{mygray},
  rulecolor=\color{cyan},
  % fancyvrb=true,
  breaklines=true
}
$endif$

$if(lhs)$
\lstnewenvironment{code}
↪ {\lstset{language=Haskell,basicstyle=\small\ttfamily}}{}
$endif$
$if(highlighting-macros)$
$highlighting-macros$
$endif$
```

```
$if(verbatim-in-note)$
\usepackage{fancyvrb}
$endif$
$if(fancy-enums)$
% Redefine labelwidth for lists; otherwise, the enumerate
↪  package will cause
% markers to extend beyond the left margin.
\makeatletter\AtBeginDocument{%
  \renewcommand{\@listi}
    {\setlength{\labelwidth}{4em}}
}\makeatother
\usepackage{enumerate}
$endif$
```

设置图版格式默认为图片大小，如果超出页面则自动缩放。

```
$if(tables)$
\usepackage{ctable}
\usepackage{float} % provides the H option for float
↪  placement
$endif$
\let\Oldincludegraphics\includegraphics
$if(graphics)$
\usepackage{graphicx}
% We will generate all images so they have a width
↪  \maxwidth. This means
```

```
% that they will get their normal width if they fit onto
↪  the page, but
% are scaled down if they would overflow the margins.
\makeatletter
\def\maxwidth{\ifdim\Gin@nat@width>\linewidth\linewidth
\else\Gin@nat@width\fi}
\makeatother
\let\Oldincludegraphics\includegraphics
\renewcommand{\includegraphics}[1]
↪  {\Oldincludegraphics[width=\maxwidth]{#1}}
% hack figure to position figure at Here!
\makeatletter
\renewcommand\fps@figure{H}
\makeatletter
$endif$
```

URL 相关的处理，以及其它。

```
\usepackage[hyphens]{url}
\ifxetex
  \usepackage[setpagesize=false, % page size defined by
  ↪  xetex
              unicode=false, % unicode breaks when used
↪  with xetex
              xetex]{hyperref}
\else
  \usepackage[unicode=true]{hyperref}
\fi
```

```
\hypersetup{breaklinks=true,
            bookmarks=true,
            pdfauthor={$author-meta$},
            pdftitle={$title-meta$},
            colorlinks=true,
            urlcolor=$if(urlcolor)$$urlcolor$
↪  $else$blue$endif$,
            linkcolor=$if(linkcolor)$$linkcolor$
↪  $else$magenta$endif$,
            pdfborder={0 0 0}}
$if(links-as-notes)$
% Make links footnotes instead of hotlinks:
\renewcommand{\href}[2]{#2\footnote{\url{#1}}}
$endif$
$if(strikeout)$
\usepackage[normalem]{ulem}
% avoid problems with \sout in headers with hyperref:
\pdfstringdefDisableCommands{\renewcommand{\sout}{}}
$endif$
% \setlength{\parindent}{2em}
\setlength{\parskip}{6pt plus 2pt minus 1pt}
\renewcommand{\baselinestretch}{1.4}
\setlength{\emergencystretch}{3em}  % prevent overfull
↪  lines
$if(numbersections)$
$else$
\setcounter{secnumdepth}{0}
$endif$
$if(verbatim-in-note)$
\VerbatimFootnotes % allows verbatim text in footnotes
```

```
$endif$
$if(lang)$
\ifxetex
  \usepackage{polyglossia}
  \setmainlanguage{$mainlang$}
\else
  \usepackage[$lang$]{babel}
\fi
$endif$
```

代码高亮。

```
\usepackage{fvextra}
\DefineVerbatimEnvironment{Highlighting}{Verbatim}
↪  {breaklines,commandchars=\\\{\}}
```

其它，将变量变成命令以便在正文中使用。

```
\usepackage{afterpage}

\newcommand{\thetitle}{$title$}
\newcommand{\theauthor}{$author$}
\newcommand{\theauthors}{$author$}
\newcommand{\thepublisher}{$publisher$}
```

```
% fix --listing missing passthrough https://github.com/
↪  laboon/ebook/issues/139
% \newcommand{\passthrough}[1]{\lstset{mathescape=false}
↪  \color{red}#1\color{black}\lstset{mathescape=true}}
```

命令行中指定的文件可以插入到这些位置。

```
$for(header-includes)$
$header-includes$
$endfor$

$if(title)$
\title{$title$}
$endif$
\author{$for(author)$$author$$sep$ \and $endfor$}
$if(date)$
\date{$date$}
$else$
\date{\today}
$endif$
```

文档开始。 `\cover` 画封面，还记得 `cover.tex` 吗？

```
\begin{document}
\cover
```

封面后的一页，书名和作者、出版社等。

```
\newpage
\thispagestyle{empty}
\pagenumbering{gobble}
\begin{center}
\vspace*{2cm}
$if(mobile)$
  \textbf{\huge \thetitle\\[1em]}
  \textbf{\Large \theauthors}
$else$
  \textbf{\Huge \thetitle\\[1em]}
  \textbf{\LARGE \theauthors}
$endif$
\end{center}

\bigskip
\vfill
\begin{center}
{\color{blue}\textbf\thepublisher}
\end{center}
```

用罗马数字标记页号。

```
\pagenumbering{Roman}
```

这里插入前言。

```
$for(include-before)$
$include-before$
$endfor$
```

如果有 `--toc`，则生成目录。

```
$if(toc)$
{
\cleardoublepage
\hypersetup{linkcolor=blue}
\tableofcontents
}
$endif$
```

书的正文开始，将页号改为阿拉伯数字。

```
\cleardoublepage
\pagenumbering{arabic}
```

正文，Markdown 的内容全部会转换成 Latex 格式放到这儿。

```
$body$
```

后面的参考资料之类的，我们这里没有用到。

```
$if(natbib)$
$if(biblio-files)$
$if(biblio-title)$
$if(book-class)$
\renewcommand\bibname{$biblio-title$}
$else$
\renewcommand\refname{$biblio-title$}
$endif$
$endif$
\bibliography{$biblio-files$}
$endif$
$endif$
$if(biblatex)$
\printbibliography$if(biblio-title)
↪  $[title=$biblio-title$]$endif$
$endif$

$for(include-after)$
$include-after$
$endfor$

\end{document}
```

好玩不?

—— 版权所有　　侵权必究——

第三章 我在 Mac 上的写作工具链

杜金房/2015.08.13/来自 FreeSWITCH-CN 公众号

有读者问我在写书时是如何画插图又如何生成 PDF 的，今天，就跟大家分享一下。

先讲个故事。

大约 09 年底 10 年初的时候，我就想写点关于 FreeSWITCH 的东西，也希望最后能汇集成书。因此，我从最开始就研究怎么排版。

最开始，是从第二章写起的。当时直接使用 Textmate 写，使用 Latex 语法，用 PdfLatex 转成 PDF。事实证明，写的很痛苦。因为，跟那些写小说的人不同，我的书中有好多代码。大段的代码还好说，直接嵌入段落中

的代码就比较麻烦，必须用一堆 $$ 括起来，而且，如果代码中有 $ 就得用 \$ 转义。不幸的是，我最开始写的那一章中有很多的 $ 。

后来，发现写书是很遥远的事，就开始把内容放到我的博客上。博客是自己建的，使用简单的 Markdown 格式。Markdown 写起来很方便，而且兼容 HTML，一切都好。

但我一直忘不了寻求如何生成 PDF 的方案。也尝试过 RestructuredText 等方案，但都不完美。

再后来，一个自称是 Tim Yang 的网友鼓励我把书印出来。排版的事就又提上了日程。由于我时间不多，一个老朋友答应帮忙。后来，还真让她整理出了一个 Word 版。但是，问题是 Word 版里有很多地方需要改，而修改 Word 版，不仅是很累的活，而且，无法同步回 Markdown，并且我也不怎么喜欢（极其讨厌）Word。所以，我只好把那么精心打造的 Word 版给废了。再次寻找从 Markdown 生成 PDF 的方案。很幸运，我找到了 Pandoc。

Pandoc 是用 Haskell 编写的软件，能从各种文件格式中互转。比较典型的就是从 Markdown 转成 PDF、HTML 和 Word。

当然，它还是先将 Markdown 转成 Latex，再从 Latex

转成 PDF。通过精心调校以及一些 Shell 脚本的帮助（用于调整 Latex），终于能生成比较美观的 PDF 了。书也印了出来，名为《FreeSWITCH：VoIP 实战》。在第一届 FreeSWITCH 沙龙上卖出去几本，后来又间或有人来买，书也一直在更新。

但，PDF 比较美观，跟完美还差一个数量级。所以，我也一直在改进。当然，在过去的几年里，Pandoc 也一直在升级。

后来，我从 PdfLatex 转成了 XeLatex，效果要好很多（具体差别细节不记得了）。我的书中有很多脚注，写起来非常方便。而且，在使用中，我也慢慢学会了如何通过在 Markdown 中适当地嵌入一些 Latex 标记来做交叉链接。现在，除了还有下列两个问题外，已经几近完美了。

1）我还没有搞定字体（我希望某些引用使用楷体，不知道为什么使用楷体后有些副作用），好在我可以通过改变字号变相的区别。

2）图文混排控制力较弱。如果不在 Markdown 中增加 Latex 标记（我不想增加），就只能实现简单的图文混排，即图片永远占一行，而无法使用文字环绕。这种情况的一个最大问题就是一个竖版的图片，往往会占一整页。如果不想让它占一整页，只好修改图片，用白色或无色背景加宽。好在我会点 PS。

同样的 Markdown 也可以生成 ePub 格式，从 ePub 可以继续生成 Kindle 格式。这两种格式与 PDF 格式比起来，还不是很完美（因为 PDF 有 Latex 帮助，而 ePub 版不能利用 Latex 标签，如果需要更好的 ePub，需要一些相关的插件才行），但是，一般来说也够用了，在移动设备上有比较好的阅读体验。

不过，即使你的 Latex 和 PDF 再精美，出版社也不会用的。后来，出书的时机成熟后我发现，我的出版社只接受 Word 格式的稿子。他们说中国的环境不足以养一个会 Latex 的编辑。另外，Word 的修改和批注功能确实没有什么很好的替代品。我虽然跟他们说『我们可以使用 Git 啊』，但他们只是笑笑。其实我自己也知道，Git 是基于行的，不适合 diff 大段的文本（更别提中文单词和标点之间没有空格了）。

所以，我只好将我的 Markdown 通过 Pandoc 转成 Word 格式。再跟编辑们一遍遍地校对和修改。当然，这些修改就只存在于 Word 版了。除非手工同步修改 Markdown，不可能再从 Word 版转回原来的 Markdown。好在，书出版以后，我不需要再继续打印了。因此，也没有必要再保持 Markdown 跟最终的版本同步。而且，出版社给出了一个 Word 模版，通过使用一些快捷键，可以很方便地格式化一级标题二级标题以及代码块等，手工按这些快捷键处理一章也基本上就是几分钟的事（Pandoc

似乎也支持模板，但我没有试过，因为反正在编辑的时候还是要再次阅读和修改的）。

书于去年 6 月出版，出版时的名字叫《FreeSWITCH 权威指南》。

后来，在读了《Producter》一书（电子书）后，我也深受启发，在 SelfStore 上发了几本我的电子书《FreeSWITCH 互联互通》、《FreeSWITCH 实例解析》等。

当然，如果我今天只讲故事，没有干货的话，你可能觉得我是在做广告了。所以，接下来干货。

在写书时，要有很多的插图。据说 Mac 上最好的画图工具是 OmniGraffle，然而，我始终没有冲动试用它。而是尝试了很多不同的软件：Skitch、Dia、yEd、XMind、Keynote 等。这些软件的问题是画得不好看，而且风格不统一。所以，最后，我又回到了程序员神器——Graphviz。

通过 Graphviz，可以使用很简单的 dot 语言画出很复杂的图，比如今天的题图，对应的 dot 源文件如下（虽然长，但很直观）：

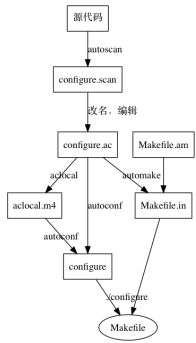
```
graph G {
    ranksep = 3;
    ratio = auto;
```

```
A [label = ""];
B [label = ""];
C [label = ""];
D [label = ""];
E [label = ""];
F [label = ""];
G [label = ""];
H [label = ""];
I [label = ""];
J [label = ""];

A -- B;
A -- C;
A -- D;
A -- E;
A -- F;
A -- G;
A -- H;
A -- I;
A -- J;
B -- C;
B -- D;
B -- E;
B -- F;
B -- G;
B -- H;
B -- I;
B -- J;
C -- D;
```

```
    C -- E;
    C -- F;
    C -- G;
    C -- H;
    C -- I;
    C -- J;
    D -- E;
    D -- F;
    D -- G;
    D -- H;
    D -- I;
    D -- J;
    E -- F;
    E -- G;
    E -- H;
    E -- I;
    E -- J;
    F -- G;
    F -- H;
    F -- I;
    F -- J;
    G -- H;
    G -- I;
    G -- J;
    H -- I;
    H -- J;
    I -- J;
}
```

又比如下图：



源文件加上空行也只有 22 行（为了在手机上减少换行，取消了缩进）：

```
digraph G {
splines = false;

S [label="源代码" shape = "box"];
SCAN [label = "configure.scan" shape = "box"];
IN [label = "configure.ac" shape = "box"];
ACLOCAL [label = "aclocal.m4" shape = "box"];
configure [shape = "box"];
MakefileAm [label = "Makefile.am" shape = "box"];
MakefileIn [label = "Makefile.in" shape = "box"];

S->SCAN [label = "autoscan"];
SCAN -> IN [label = "改名，编辑"];
```

```
IN -> ACLOCAL [ label = "aclocal"];
ACLOCAL -> configure [ label = "autoconf"];
IN -> configure [ label = "autoconf"];
MakefileAm -> MakefileIn;
IN -> MakefileIn [ label = "automake"];

MakefileIn -> Makefile;
configure -> Makefile [label = "./configure"];
}
```

而所有的 dot 文件，只是靠一个简单的 Makefile 和一个 make 命令就会部都转成 PNG 图片了。同时，dot 文件还可以存在 Git 仓库里。

谁能告诉我，像下面这样的图（如图3.1）用 OmniGraffle 或 Windows 上的 Visio 画累不累呢？

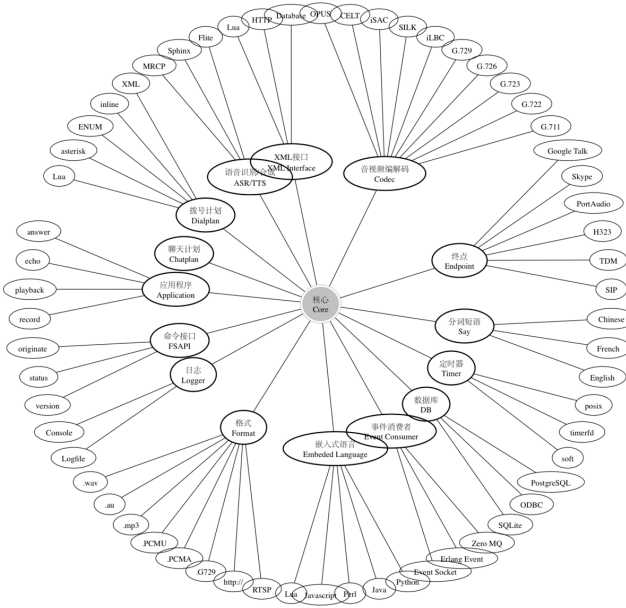


图 3.1: FreeSWITCH 模块结构

当然，并不是所有插图都适合用 Graphviz。所以，除此之外，我还使用 Keynote。Keynote 比较适合画这样的图（是的，Internet 也是我用弧线画出来的，如图3.2）：

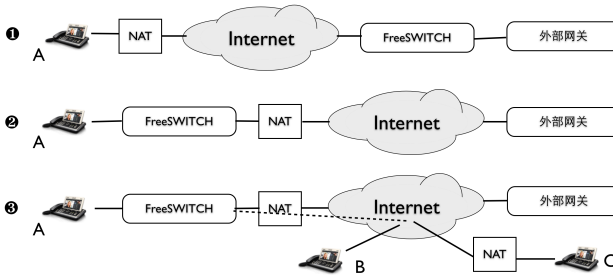


图 3.2: 用 Keynote 画的 NAT 示意图

还有一个问题。那就是，以上工具都不适合画时序图。因此，我不得不又搜遍互联网，找到一个类似 Graphviz 的工具——mscgen。是的，跟我期望的一样，它可以使用类似 dot 的语法画时序图，大概就是下面这个样子，如图3.3：

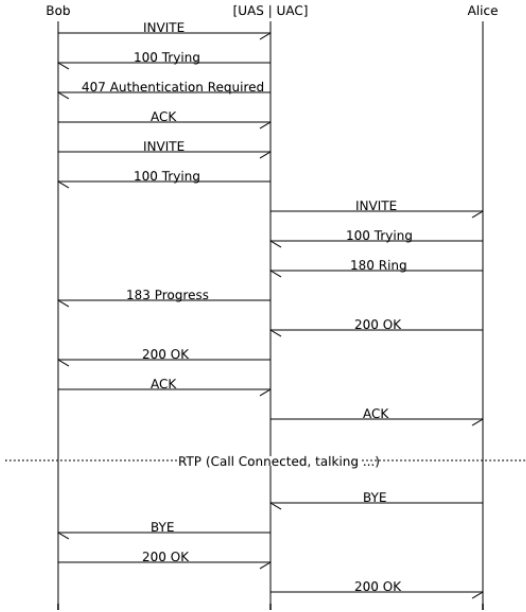


图 3.3: SIP 信令时序图

搞过通信的人应该对上面那种图都非常熟悉。当然 mscgen 在 Mac 上生成图时对中文的处理有点问题。我只好首先用 mscgen 生成 svg，然后又用 Inksape 将 svg 转换成 PNG。

你还在用 Word 写 API 文档和产品说明文档吗？反正我们已经全部是 Markdown 了。虽然 Word 也可以进行『文件比较』，但怎么也不如 `git diff` 来得顺手吧？

好了。这基本上就是全部的秘密了。如果大家感兴趣，我以后也可以跟大家分享一些相关的脚本和源文件。程序员万岁。

—— 版权所有　　侵权必究——

写在最后

到此，我们该写一个后记了。如果你查看源文件，你可以看到从这里开始的相关的章节名字后面有 `{-}` 标记，它告诉 LaTex 按章节排版但不再生成章节号。

本书所有源代码都可以在 Github 上找到：https://github.com/seven1240/latex。

用 LaTex 排版，你几乎可以排出任何你想要的效果。但是，我们在本书中只使用了基本的 LaTex 的模板，在表格、图文混排方面还有很多不尽人意。不过，我们的目标并不是做一个完美的排版，因为那样需要在正文中插入很多跟内容不相关格式代码，就背离了我们想使用 Markdown 把文章写的简单清新的初衷了。所以，我们只是尽力而为，做一个『足够好看』的电子书即可。

如果你的书真的要出版，出版社会帮你排版的。

LaTex 的学习曲线也是很陡的，而且在 LaTex 中处理中

文，就需要更多的技巧。所幸，现在中文都统一的 UTF-8 编码了，所有一定保证所有源文件都保存成 UTF-8 的。如前言中所述，本书不是希望教你成为一个排版专家，这些模板已经写好了，直接拿去用就可以了。当然，如果你真是个排版专家，也欢迎提到 Github 上提 PR 帮我们做得更好。

我们还有以下问题没有解决：

- 我不知道怎么在正文中自由切换字体，如临时变成楷体、仿宋等。
- 表格，我想默认设成 100% 宽度。

这是	一张表格
内容	太短了不好看
不知	有没有办法宽度 100%

作者简介

杜金房：（网名：Seven Du），《FreeSWITCH 权威指南》[1]作者、FreeSWITCH 中文社区[2]创始人，FreeSWITCH 开源项目[3]核心 Committer，开源爱好者。北京信悦通科技和烟台小樱桃科技[4]创始人。

[1] http://book.dujinfang.com，2014 年出版。

[2] http://www.freeswitch.org.cn

[3] https://freeswitch.com

[4] http://x-y-t.cn

版权声明

本书版权归作者所有，保留所有权利。本书相关的模板代码采用创作共用 CC-BY-SA[5]发布。

[5] https://zh.wikipedia.org/wiki/Wikipedia:CC_BY-SA_3.0 协议文本>

广告

关于广告的广告

请允许我在本书中发布广告。承接其它广告。广告合作联系邮箱：info@x-y-t.cn 。

XSWITCH

我自己的通信助手 https://xswitch.cn

XSWITCH 是一个 SSaaS（Soft-Switch as a Service）平台，可以用来：

- 打电话
- 电话会议

- 视频会议
- 呼叫中心
- 录音录像
- 其它音视频互通等

支持私有化部署。

技术支持

烟台小樱桃网络科技有限公司提供商业 FreeSWITCH、OpenSIPS 及 Kamailio 技术支持。

- 网址：http://x-y-t.cn
- 邮箱：info@x-y-t.cn

图 3.4: 小樱桃科技微信公众号

FreeSWITCH-CN 公众号

图 3.5: FreeSWITCH-CN 微信公众号

FreeSWITCH 相关图书

- 《FreeSWITCH 文集》收集了一些 FreeSWITCH 文章，相比其它 FreeSWITCH 书来说，技术内容比较少，便于非技术人员快速了解 FreeSWITCH。
- 《FreeSWITCH 互联互通》主要收集了一些互联互通的例子。书中有些例子来自《FreeSWITCH 权威指南》。
- 《FreeSWITCH 实例解析》收集了一些如何使用 FreeSWITCH 的实际例子，方便读者参考。书中有些内容来自《FreeSWITCH 权威指南》。
- 《FreeSWITCH 实战》是《FreeSWITCH 权威指南》

的前身，不再更新，但该书有其历史意义。

- 《FreeSWITCH WIRESHARK》是一本介绍如果用 Wireshark 分析 SIP/RTP 数据包的书。
- 《FreeSWITCH 源代码分析》，讲源代码。
- 《FreeSWITCH 权威指南》是正式出版的纸质书和电子书。

以上所有图书均可以在http://book.dujinfang.com 查看最新信息及购买。FreeSWITCH VIP 知识星球里面也有部分电子书。

知识星球

杜老师维护着两个知识星球，一个免费版，一个收费版。可以使用如下链接或通过微信扫描二维码加入。

- FreeSWITCH：https://t.zsxq.com/RBi6Ee2
- FreeSWITCH VIP：https://t.zsxq.com/2zb6qBE

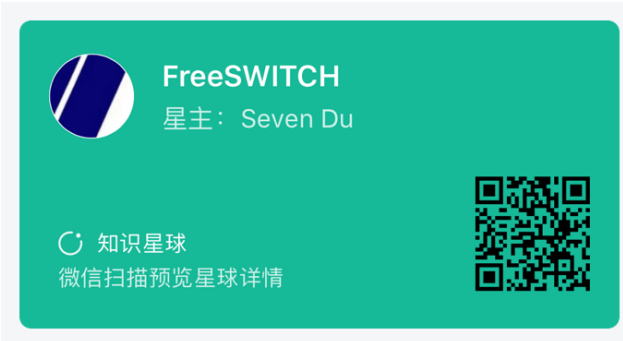


图 3.6: 知识星球 FreeSWITCH

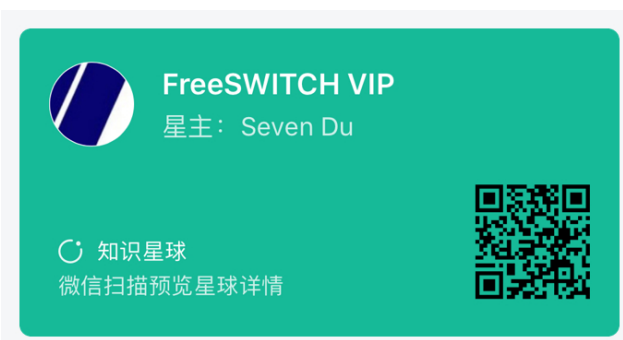


图 3.7: 知识星球 FreeSWITCH VIP

THIS PAGE INTENTIONALLY LEFT BLANK.